

Robust Neural Learning via Divergences — Research Understanding

Intuition:

All three papers solve the same problem: standard neural networks use the KL divergence (= cross-entropy or MSE) as their loss. KL divergence is sensitive to even just one bad data point. The papers replace it with a more forgiving divergence that automatically “soft-ignores” suspicious data points during training — without having to explicitly find or remove those bad points.

The papers also talk about: (a) which kind of “bad data” is being tolerated, (b) which divergence family is being used, and (c) which type of neural network (regression vs. classification).

Paper 1: **β -divergences.pdf**

“Provably robust learning of regression neural networks using β -divergences”

— Abhik Ghosh & Suryasis Jana, ISI Kolkata. *arXiv:2602.08933*, February 2026.

What problem does it solve?

Setting: Regression neural network (predicts a continuous number, like predicting house price or sensor reading from features) we train it with mean squared error.

The bug: MSE sums up $(\text{prediction} - \text{true_value})^2$ for all data points. If even one data point has a corrupted label (e.g., 99999 instead of 99), that squared error completely dominates the training and destroys your model. This is called **non-robustness to label noise / data contamination**.

Current solutions are weak: Methods like clipping, winsorizing, or Huber loss help a little, but either have no theoretical guarantees, or are limited in scope, or require you to manually tune thresholds to flag outliers.

Claim of recent papers:

rRNet (robust Regression Network): swap out MSE for the **β -divergence (Density Power Divergence or DPD)** as the training objective.

The β -divergence between the true data distribution g and the model distribution f is:

$$d_{\beta}(g, f) = \int f^{1+\beta} - \left(1 + \frac{1}{\beta}\right) f^{\beta} g + \frac{1}{\beta} g^{1+\beta} d\lambda$$

The tuning parameter $\beta \geq 0$ controls the trade-off: - $\beta = 0 \rightarrow$ reduces exactly to KL divergence = standard MLE = standard MSE training (*no robustness*) - $\beta > 0 \rightarrow$ introduces robustness; larger β = more robust but slightly less efficient on clean data - **Practical sweet spot: $\beta \in (0.1, 0.5)$** — provides strong robustness with minimal accuracy loss on clean data

Key insight: The **β -divergence** loss function for Gaussian errors (example) becomes:

$$L_{n,\beta}(\theta, \sigma) = \frac{1}{(\sigma\sqrt{2\pi})^\beta \sqrt{1+\beta}} - \frac{1+\beta}{\beta} \cdot \frac{1}{n(\sigma\sqrt{2\pi})^\beta} \sum_{i=1}^n e^{-\frac{\beta}{2\sigma^2}(y_i - \mu(x_i, \theta))^2} +$$

Notice: each data point i contributes an **exponentially weighted** term. If $(y_i - \text{prediction})^2$ is huge (outlier), the exponential $e^{-\beta \times \text{huge}}$ goes to near-zero $\rightarrow$ that data point barely influences the gradient. **No explicit outlier detection. It's automatic.**

How is it trained? (The Algorithm)

Because you're simultaneously estimating θ (network weights) and σ (error scale), they use **alternating optimization**:

1. Fix σ , update θ via gradient descent on the **β -divergence** loss
2. Fix θ , update σ via a closed-form formula
3. Repeat until convergence

They prove convergence of this alternating scheme to stationary points — this is a non-trivial theoretical contribution because the DPD loss is non-convex and works even with non-smooth activations like ReLU.

What are the theoretical guarantees?

Three big ones you can remember as “**BIB**”:

1. **Bounded Influence Function** (local robustness): The influence function measures “how much does one slightly perturbed data point change the learned model?” For $\beta > 0$, this is bounded — one bad point can only push the model a bounded amount. For $\beta = 0$ (standard MSE), it is unbounded.
2. **Influence Function shape is redescending**: Not just bounded, but the influence goes back *toward zero* for extreme outliers — meaning the model actively ignores very extreme corruptions.

3. **Breakdown Point** = 50% (global robustness): The **breakdown point** is the fraction of data that can be corrupted before the model estimate completely fails. They prove rRNet achieves the optimal **50% asymptotic breakdown point** for all $\beta \in (0,1]$. This means: up to half your training data can be corrupted and the model still works! Standard MSE training has a breakdown point of essentially 0% (one extreme outlier is all it takes).
-

How to remember Paper 1:

“Regression + MSE is fragile. Swap MSE for DPD (β -divergence). One β knob. $\beta=0$ is MSE. $\beta>0$ is robust. Provably tolerates up to 50% corrupted data.”

Paper 2: `s_divergence.pdf`

“A generalized divergence for statistical inference”

— Abhik Ghosh, Ian Harris, Avijit Maji, A. Basu, L. Pardo. *Bernoulli* 23(4A), 2017.

Why does this paper exist?

Before rSDNet (Paper 3), the world had two families of robust divergences: 1. **Power Divergence (PD)** family — great for discrete models, but requires kernel density estimation for continuous data (messy, bandwidth selection problem) 2. **Density Power Divergence (DPD)** family — the β -divergence from Paper 1 — clean, closed-form, no kernel smoothing needed

The problem discovered: In some contamination scenarios, the best estimator lies **neither** in PD nor DPD. There is a “gap” between them. These families are 1-dimensional curves in the space of possible divergences, and the best divergence might be somewhere in the 2D plane between them.

The solution: This paper introduces the **S-divergence**, a 2-parameter **superfamily** that contains both PD and DPD as special cases:

$$S_{(\alpha,\lambda)}(g, f) = \frac{1}{A} \int f^{1+\alpha} - \frac{1+\alpha}{AB} \int f^B g^A + \frac{1}{B} \int g^{1+\alpha}$$

where **A** = **1 + λ(1-α)** and **B** = **α - λ(1-α)**.

Special cases (memorize this table):

α	λ	What it gives
0	any	Power Divergence (PD) with parameter λ
any	0	Density Power Divergence (DPD) with parameter α — same as β-divergence!
1	any	Squared L ² distance (regardless of λ)
0	-1	KL divergence (= standard cross-entropy = standard training)

So the **full S-divergence landscape** is a 2D surface, and: - The **left edge** (α=0) is PD - The **vertical center** (λ=0) is DPD/β-divergence - The **top edge** (α=1) is L² distance - The **origin corner** (α=0, λ=-1) is KL / cross-entropy / MSE

The key empirical finding

In simulation experiments with contaminated data (e.g., Poisson model with 10% outliers from a heavy distribution), the best-performing estimator was at, say, **(α=0.4, λ=-0.7)** — which is **NOT on the PD axis** and **NOT on the DPD axis**. It's in the interior of the S-divergence family.

This proves the S-divergence isn't just a routine mathematical exercise — it has **real utility** that PD and DPD cannot provide.

The finding about influence functions

A curious result: the **first-order influence function** of the MSDE (Minimum S-Divergence Estimator) is **independent of λ** ! This means standard robustness analysis (which uses influence functions) cannot tell the difference between two S-divergence estimators with the same α but different λ values — even though they perform very differently in practice.

This is a theoretical warning: **standard influence function analysis is insufficient** to characterize S-divergence robustness. You need second-order or breakdown-point analysis.

How to remember Paper 2:

“S-divergence is the 2-parameter family that unifies PD and DPD. Think of it as a 2D map where DPD (β -divergence) is a road running through the middle. The best route through contaminated terrain might be off that road. (α, λ) are the GPS coordinates.”

Paper 3: [rSDNet.pdf](#)

“rSDNet: Unified Robust Neural Learning against Label Noise and Adversarial Attacks”

— Suryasis Jana & Abhik Ghosh, ISI Kolkata. *arXiv:2603.17628*, March 2026.
[THE KEY PAPER for your collaboration]

What problem does it solve?

Setting: Classification neural network (output: probabilities over K classes via softmax, trained with cross-entropy / CCE loss).

Two problems CCE doesn't handle:

1. **Label Noise:** A fraction of training examples have wrong labels (e.g., an image of a cat labeled “dog”). CCE trains confidently on everything including these corrupt labels → model memorizes noise.
2. **Adversarial Attacks:** An adversary adds a tiny imperceptible perturbation to an input image at inference time that completely flips the model's prediction (e.g., FGSM, PGD attacks). CCE-trained models are famously brittle to these.

Current approaches treat these separately: - Label noise: use noise-robust losses (symmetric CE, generalized CE, truncated CE, etc.) - Adversarial attacks: use adversarial training (generate adversarial examples and include them in training), certified defenses, etc.

The insight: Both label noise and adversarial perturbations are forms of **data contamination** — they corrupt either the output space (labels) or the input space (features). A sufficiently robust loss function should handle both within a single training objective.

What do they propose?

Replace CCE loss with the **S-divergence loss**:

For classification, the standard CCE loss is:

$$L_0(\theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^J y_{ij} \log p_j(x_i; \theta)$$

The **rSDNet loss** replaces this with the empirical SD-risk:

$$L_{\beta, \lambda}^{(n)}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_{\beta, \lambda}(y_i, p(x_i; \theta))$$

where the per-sample SD-loss (for the full S-divergence) is:

$$\ell_{\beta,\lambda}(y, p) = \frac{1}{A} \sum_{j=1}^J p_j^{1+\beta} - \frac{1+\beta}{AB} \sum_{j=1}^J p_j^B y_j + \frac{A}{B} \sum_{j=1}^J y_j^{1+\beta}$$

with $A = 1 + \lambda(1-\beta)$ and $B = \beta - \lambda(1-\beta)$, and the final term is a constant w.r.t. θ .

Two parameters: (β, λ) : - $\beta \in (0.05, 0.3)$ for classification: small values work well — classification is harder than regression so less robustness budget needed - $\lambda \in (-1, -0.5)$: negative λ with small positive β gives the best practical robustness (matches the S-divergence paper’s empirical finding) - At $\beta=0$, $\lambda=-1$: reduces exactly to CCE (standard training)

The mechanism of robustness — intuitively

In CCE: gradient of loss for sample $i \propto (y_i - p(x_i; \theta)) \rightarrow$ a mislabeled sample always contributes a full gradient, actively misleading training.

In rSDNet: the gradient gets multiplied by $p_j(x_i; \theta)^\beta$ — the model’s own confidence.

If the model is confident but label is wrong (adversarial/mislabeled):

model confidence p_j ends up high for the wrong class $\rightarrow$ the corrupted gradient gets *amplified if trusted* but *the SD-loss structure down-weights the entire contaminated sample*. This is the self-protective mechanism.

Think of it as: “**the loss function looks at itself (the model’s probabilities), and uses that to decide how much to trust each training point.**”

Theoretical guarantees

Four results, remember as “**FCRB**”:

1. **Fisher Consistency:** At the population level, rSDNet’s minimizer gives exactly the true class probabilities $p^*(x)$ — it’s an unbiased estimator. Not true for MAE loss or other ad-hoc robust losses.

2. **Classification-Calibrated:** The rSDNet loss is *classification-calibrated* — meaning even though you're minimizing a custom loss, the resulting classifier still achieves Bayes-optimal class predictions. This is a deep property (not all robust losses satisfy it).
 3. **Robustness to uniform label noise:** Theorem 3.3 shows that under uniform noise with rate η , the expected SD-risk under noise is a **monotone function** of the clean SD-risk $\rightarrow$ rSDNet trained on noisy data still converges to a classifier close to the Bayes optimum.
 4. **Bounded influence under adversarial perturbation:** The influence function for infinitesimal adversarial input contamination is bounded for $\beta > 0$ (and is the same structure as in Paper 1 for regression).
-

Experiments (the empirical story)

Tested on **MNIST**, **Fashion-MNIST**, **CIFAR-10** against these baselines: - **CCE** (standard cross-entropy) - **MAE** (mean absolute error loss) - **GCE** (Generalized Cross-Entropy, Zhang & Sabuncu 2018) - **SCE** (Symmetric Cross-Entropy) - **TCCE** (Truncated Cross-Entropy) - **FCL** (Focal Loss) - **rKLD** (robust KLD variant)

Summary of results:

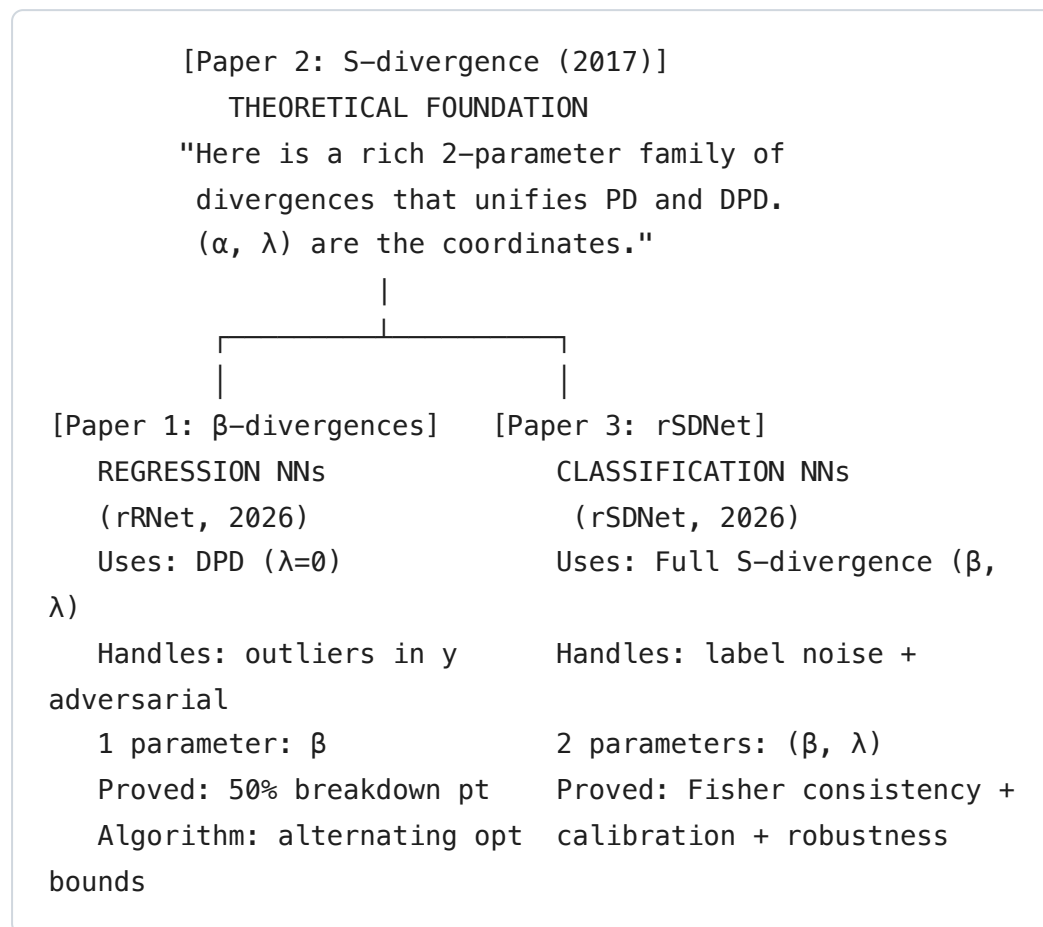
Scenario	Winner
Clean data	CCE $\approx$ rSDNet (no penalty for using rSDNet)
10–30% label noise	rSDNet best or co-best across all datasets
40–50% label noise	rSDNet clearly dominant for CIFAR-10; competitive on others
MAE on CIFAR-10 (complex)	Completely collapses to 10% accuracy
rSDNet on CIFAR-10 noisy	Remains strongest across all contamination levels

The **CNN architecture was used for CIFAR-10**, showing rSDNet works with convolutional models (not just MLP). This is the jumping-off point for your collaboration.

How to remember Paper 3:

“Classification + CCE is fragile to label noise AND adversarial attacks. rSDNet swaps CCE for S-divergence loss. Two knobs (β, λ). Down-weights mislabeled/adversarial samples using model’s own probabilities. Theoretically sound: Fisher consistent, Bayes optimal, robust. Tested on MNIST/FashionMNIST/CIFAR-10 with MLP and CNN. Code in the git repo for MLP. Your collaboration extends this to CNN and Transformer architectures.”

The Grand Unifying Picture: How All Three Papers Connect



The academic lineage: - Abhik Ghosh co-authored all three papers - Paper 2 (Bernoulli 2017) = the mathematical theory - Papers 1 & 3 (arXiv 2026) = the neural network applications - Paper 3 is what the code implements and what your collaboration extends

The progression of generality: - KL divergence (=CCE, =MSE) → 1D: β -divergence → 2D: S-divergence - One-parameter robustness → Two-parameter robustness (strictly more powerful)

Concepts That Are New To You: A Quick Glossary

Data Contamination: Your training data has some percentage of bad samples. Could be: wrong labels (annotation errors, adversarial examples), corrupted features (sensor noise, JPEG artifacts), or outliers in the target values.

Label Noise / Noisy Labels: Some fraction η of training labels are wrong. Types: - *Uniform noise*: wrong label is randomly chosen from other classes (Paper 3 focuses here) - *Asymmetric noise*: specific classes get confused with specific other classes (e.g., cat $\leftrightarrow$ dog) - *Instance-dependent noise*: harder-to-classify examples more likely to have wrong labels

Influence Function: A mathematical tool borrowed from statistics (Hampel 1974). Measures: “if I add one infinitesimally weighted contaminated point to my training data, how much does my model estimate change?” Bounded IF = robust. Unbounded IF = one bad point can destroy everything.

Breakdown Point: The maximum fraction of arbitrarily bad data points the estimator can tolerate before completely “breaking down” (giving useless predictions). 50% is the theoretical maximum possible. rRNet achieves this.

Minimum Divergence Estimation (MDE): A classical statistics framework. Instead of maximizing likelihood (MLE), you minimize a divergence measure between your empirical data distribution and your parametric model. MLE = minimizing KL divergence. rSDNet = minimizing S-divergence.

Fisher Consistency: A sanity check for estimators. Means: if you had infinite clean data from the exact model, the estimator exactly recovers the true parameters. All good estimators must satisfy this.

Classification Calibration: A property of loss functions that guarantees: minimizing this loss leads to Bayes-optimal classification decisions (assigns the correct predicted class). Not all robust losses have this property.

Adversarial Attacks (FGSM, PGD): - *FGSM (Fast Gradient Sign Method)*: adds $\epsilon \times \text{sign}(\nabla_x L)$ to an image — moves the input a tiny step in the direction that maximally increases the loss - *PGD (Projected Gradient Descent)*: iterated FGSM steps, staying within an ϵ -ball — stronger attack

Why This Should Excite You (CS + AI Safety Perspective)

Connection to AI Safety & Alignment

Even though the group frames this in classical statistics, the problems they solve are **core to AI safety**:

rSDNet topic	AI Safety equivalent
Robustness to label noise	Robustness to unreliable human feedback in RLHF (reward hacking via mislabeled preferences)
Robustness to adversarial attacks	Adversarial robustness — making models that can't be fooled by malicious inputs
Bounded influence functions	Stability guarantees — formal bounds on how much any single data point can corrupt the model
50% breakdown point	Data poisoning resilience — attacker must control >50% of data to break the system
Fisher consistency	Alignment — model converges to the correct answer in the limit

The S-divergence as a “Loss Function Design Space”

From a deep learning perspective, what these papers do is:

1. **Standard training:** CCE loss = KL divergence. Fixed. No freedom.
2. **rSDNet:** You get a 2D space of loss functions parameterized by (β, λ) . Each point in this space has different robustness properties.
3. **The research insight:** Finding the right (β, λ) for a given dataset/corruption type is a learnable, principled problem.

Future direction: Can β and λ be **learned** from data (meta-learned or per-sample adaptive) rather than treated as fixed hyperparameters?

The Architecture Extension — Your Role

The rSDNet paper already uses CNN for CIFAR-10. The existing GitHub code is MLP-only. The collaboration wants to extend to: - **CNN architectures** (convolutional blocks + the SD-loss as the final loss) - **Transformer architectures** (ViT, BERT-style, with SD-loss instead of CCE)

Why this is technically straightforward to implement: The SD-loss is just a different function of `p(x;θ)` (the softmax output). The loss function is decoupled from the architecture. You can swap out `CrossEntropyLoss()` with the `SDLoss(beta, lambda)` in any PyTorch/TensorFlow code. The backpropagation through the architecture does not change.

The open research questions your group is positioned to answer: 1. Does rSDNet’s robustness scale to larger Transformer models and harder datasets (ImageNet, text classification)? 2. How does the optimal (β, λ) vary across architectures and dataset corruption levels? 3. Can rSDNet robustness combine with adversarial training for compounded benefits? 4. What happens with **instance-dependent** label noise (not just uniform)? The theory covers uniform noise; the harder case is still open. 5. Is there a way to adapt (β, λ) per-sample? (“Curriculum of robustness”)

The GitHub Code: What It Does and What You’ll Add

Repo: <https://github.com/Suryasis124/Robust-NN-learning>

What’s there (MLP for rSDNet):

```
# Conceptually, the core change is replacing:
loss = F.cross_entropy(logits, labels)

# With:
loss = sd_loss(softmax_probs, one_hot_labels, beta=0.1,
               lambda_=-0.5)
```

where `sd_loss` computes:

```
def sd_loss(p, y_onehot, beta, lam):
    A = 1 + lam * (1 - beta)
    B = beta - lam * (1 - beta)
    term1 = (1/A) * torch.sum(p**(1+beta), dim=1)
    term2 = ((1+beta)/(A*B)) * torch.sum(p**B *
                                           y_onehot**A, dim=1)
    # term3 is constant w.r.t. theta, can be dropped for
    # optimization
    return (term1 - term2).mean()
```

What you'll add: 1. Wrap this loss into standard CNN training loops (ResNet, EfficientNet blocks) 2. Plug into Transformer training (ViT, BERT fine-tuning) 3. Reproduce their experiments with your architectures 4. Benchmark: does robustness scale? Is the gain larger for more expressive architectures? 5. Possibly: adaptive (β , λ) scheduling during training

How To Write To Them: Key Points to Convey

Suggested message structure for your email to Subho, Anand, and Abhik:

- Acknowledge the core framework:** "I understand that rSDNet replaces cross-entropy (= KL divergence $\rightarrow$ minimum KL estimation) with the S-divergence family parameterized by (β, λ) , giving a 2D generalization of DPD. The β -divergence paper (rRNet) does the same for regression. Both are grounded in the theoretical S-divergence superfamily from the 2017 Bernoulli paper."
- Show you understand the mechanism:** "The robustness arises because the SD-loss down-weights the gradient contribution of each sample

proportionally to p_j^β — observations the model is uncertain about OR that have inconsistent labels get automatically discounted, with no explicit outlier detection needed.”

3. **Show you understand the theory gaps:** “The proofs cover uniform label noise and infinitesimal contamination (influence functions). Instance-dependent noise and finite-sample adversarial robustness seem like natural extensions.”
 4. **Propose your extension angle:** “Since the loss function is architecture-agnostic (it only depends on the softmax output), extending rSDNet to CNNs and Transformers is a modular addition. I can implement the SD-loss as a drop-in PyTorch module and run experiments on ViT/ResNet architectures on CIFAR-10/ImageNet-subset.”
 5. **Connect to your AI-safety interest (optional):** “I also see connections between rSDNet’s robustness guarantees and AI safety concerns like data poisoning and reward hacking under noisy human feedback — this might be an interesting future direction for the collaboration.”
-

Summary Cheat Sheet (For Quick Reference)

	Paper 1 (β -divergences)	Paper 2 (S-divergence)	Paper 3 (rSDNet)
Task	Regression NN	Statistical inference (theory)	Classification NN
Standard method	MSE / LSE training	MLE / KL divergence	CCE (cross-entropy)
Problem	Sensitive to outliers in labels	PD and DPD miss optimal estimators in 2D	Sensitive to label noise + adversarial
Proposed method	rRNet (β -divergence loss)	S-divergence family (α, λ)	rSDNet (S-divergence loss)
Parameters	$\beta \in (0, 1]$	$\alpha \in [0, 1], \lambda \in \mathbb{R}$	$\beta \in (0, 1], \lambda \in \mathbb{R}$
Algorithm	Alternating opt. (θ and σ)	Iterative BFGS/gradient	SGD/Adam with SD-loss
Key theorem	50% breakdown point	IF is λ -independent (need 2nd order)	Fisher consistency + calibration
Datasets	Simulated + real regression	Poisson, Normal, Mixture	MNIST, Fashion-MNIST, CIFAR-10
Architecture	MLP (regression)	N/A	MLP + CNN
Code	Not in repo	N/A	In GitHub repo (MLP only)
Your extension	—	—	CNN + Transformer

Write-up prepared: March 2026. Based on direct reading of: [β-divergences.pdf](#) (arXiv:2602.08933), [s_divergence.pdf](#) (Bernoulli 2017), [rSDNet.pdf](#) (arXiv:2603.17628).